Exercise 1: Data Types and Conditional Statements

Aim

To calculate a student's average marks, grade, pass/fail status, and performance.

Algorithm

1. Start the program.
2. Read student name, roll number, and three subject marks.
3. Calculate the average.
4. Check whether all marks are at least 50.
5. Assign a grade based on the average.
6. Display the student's details, grade, result, and performance.
7. Stop.

Program

TURBO C++ • PROGRAM

```
#include <iostream.h>
#include <conio.h>

void main() {
    char name[30];
    int roll;
    float m1, m2, m3, avg;
    char grade;

    clrscr();
    cout << "Enter name: ";
    cin.getline(name, 30);
    cout << "Enter roll number: ";
    cin >> roll;
    cout << "Enter three marks: ";
    cin >> m1 >> m2 >> m3;

    avg = (m1 + m2 + m3) / 3;

    if (m1 < 50 || m2 < 50 || m3 < 50)
        grade = 'F';
    else if (avg >= 90)
        grade = 'A';
    else if (avg >= 80)
        grade = 'B';
    else if (avg >= 70)
        grade = 'C';
    else if (avg >= 60)
        grade = 'D';
    else
        grade = 'E';

    cout.setf(ios::fixed);
    cout.precision(2);
    cout << "Name: " << name << "\nRoll: " << roll;
    cout << "\nAverage: " << avg;
    cout << "\nGrade: " << grade;

    if (grade == 'F')
        cout << "\nResult: FAIL\nNeeds Improvement";
    else {
        cout << "\nResult: PASS";
        if (avg >= 85)
            cout << "\nExcellent";
        else if (avg >= 70)
            cout << "\nVery Good";
        else
            cout << "\nGood";
    }
    getch();
}
```

Output

TERMINAL • OUTPUT (sample input shown as typed)

```
Enter name: Anitha
Enter roll number: 101
Enter three marks: 85 78 92
Name: Anitha
Roll: 101
Average: 85.00
Grade: B
Result: PASS
Excellent
```

Result

Thus, the program calculates the average, grade, pass/fail status, and performance successfully.

Exercise 2: Finding Complexity of a Simple Algorithm

Aim

To find the maximum element in an array and count the comparisons.

Algorithm

1. Start and read the array size and elements.
2. Initialize maximum with the first element.
3. Compare each remaining element with maximum.
4. Update maximum when a larger element is found.
5. Display the maximum, comparisons, and complexity.
6. Stop.

Program

TURBO C++ • PROGRAM

```
#include <iostream.h>
#include <conio.h>
void main() {
    int n, i, a[50], big, count = 0;
    clrscr();
    cout << "Enter number of elements: ";
    cin >> n;
    cout << "Enter elements: ";
    for (i = 0; i < n; i++)
        cin >> a[i];
    big = a[0];
    for (i = 1; i < n; i++) {
        count++;                // one comparison per element
        if (a[i] > big)
            big = a[i];
    }
    cout << "Maximum: " << big;
    cout << "\nComparisons: " << count;
    cout << "\nTime Complexity: O(n)";
    cout << "\nSpace Complexity: O(1)";
    getch();
}
```

Output

TERMINAL • OUTPUT (sample input shown as typed)

```
Enter number of elements: 6
Enter elements: 25 78 12 95 43 61
Maximum: 95
Comparisons: 5
Time Complexity: O(n)
Space Complexity: O(1)
```

Result

Thus, the maximum element and number of comparisons are found successfully.

Exercise 3: Stack Using Array and Linked List

Aim

To implement stack operations using an array and a linked list.

Algorithm

A. Stack using array

1. Start and set top = -1.
2. Display the menu: Push, Pop, Peek, Display, and Exit.
3. Push: if top = MAX - 1, report overflow; otherwise increase top and store the element.
4. Pop: if top = -1, report underflow; otherwise remove the element and decrease top.
5. Peek: display the element at top.
6. Display: print elements from top down to 0.
7. Repeat until Exit is selected.

B. Stack using linked list

1. Start and set top = NULL.
2. Display the menu: Push, Pop, Peek, Display, and Exit.
3. Push: create a node, point it to top, and make it the new top.
4. Pop: if top is NULL, report underflow; otherwise move top to the next node and delete the old one.
5. Peek: display the data of the top node.
6. Display: traverse from top to the end of the list.
7. Repeat until Exit is selected.

Program A – Stack Using Array

TURBO C++ • PROGRAM

```
#include <iostream.h>
#include <conio.h>
#define MAX 5
int stack[MAX];
int top = -1;
void push(int x) {
    if (top == MAX - 1)
        cout << "Stack Overflow";
    else {
        top++;
        stack[top] = x;
        cout << "Pushed " << x;
    }
}
void pop() {
    if (top == -1)
        cout << "Stack Underflow";
    else {
        cout << "Popped " << stack[top];
        top--;
    }
}
void peek() {
    if (top == -1)
        cout << "Stack is empty";
    else
        cout << "Top element: " << stack[top];
}
void display() {
    int i;
    if (top == -1)
        cout << "Stack is empty";
    else {
        cout << "Stack (top to bottom): ";
        for (i = top; i >= 0; i--)
            cout << stack[i] << " ";
    }
}
void main() {
    int ch, x;
    clrscr();
    do {
        cout << "\n1.Push 2.Pop 3.Peek 4.Display 5.Exit";
        cout << "\nEnter choice: ";
        cin >> ch;
        switch (ch) {
            case 1: cout << "Enter element: "; cin >> x; push(x); break;
            case 2: pop();     break;
            case 3: peek();    break;
            case 4: display(); break;
            case 5: break;
            default: cout << "Invalid choice";
        }
    } while (ch != 5);
    getch();
}
```

Output – Stack Using Array

TERMINAL • OUTPUT (sample input shown as typed)

```
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 10
Pushed 10
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 20
Pushed 20
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 30
Pushed 30
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 4
Stack (top to bottom): 30 20 10
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 3
Top element: 30
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 2
Popped 30
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 4
Stack (top to bottom): 20 10
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 5
```

Program B – Stack Using Linked List

TURBO C++ • PROGRAM

```
#include <iostream.h>
#include <conio.h>
struct node {
    int data;
    struct node *next;
};
struct node *top = 0;        // top of the stack (0 means empty)
void push(int x) {
    struct node *p;
    p = new node;
    p->data = x;
    p->next = top;           // new node points to old top
    top = p;
    cout << "Pushed " << x;
}
void pop() {
    struct node *p;
    if (top == 0)
        cout << "Stack Underflow";
    else {
        p = top;
        cout << "Popped " << p->data;
        top = top->next;
        delete p;
    }
}
void peek() {
    if (top == 0)
        cout << "Stack is empty";
    else
        cout << "Top element: " << top->data;
}
void display() {
    struct node *p;
    if (top == 0)
        cout << "Stack is empty";
    else {
        cout << "Stack (top to bottom): ";
        for (p = top; p != 0; p = p->next)
            cout << p->data << " ";
    }
}
void main() {
    int ch, x;
    clrscr();
    do {
        cout << "\n1.Push 2.Pop 3.Peek 4.Display 5.Exit";
        cout << "\nEnter choice: ";
        cin >> ch;
        switch (ch) {
            case 1: cout << "Enter element: "; cin >> x; push(x); break;
            case 2: pop();     break;
            case 3: peek();    break;
            case 4: display(); break;
            case 5: break;
            default: cout << "Invalid choice";
        }
    } while (ch != 5);
    getch();
}
```

Output – Stack Using Linked List

TERMINAL • OUTPUT (sample input shown as typed)

```
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 10
Pushed 10
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 20
Pushed 20
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter element: 30
Pushed 30
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 4
Stack (top to bottom): 30 20 10
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 3
Top element: 30
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 2
Popped 30
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 4
Stack (top to bottom): 20 10
1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 5
```

Result

Thus, the stack operations are performed successfully using an array and a linked list.

Exercise 4: Queue Using Array and Linked List

Aim

To implement queue operations using an array and a linked list.

Algorithm

A. Queue using circular array

1. Start and set front = 0, rear = -1, count = 0.
2. Display the menu: Enqueue, Dequeue, Front, Rear, Display, and Exit.
3. Enqueue: if count = MAX, report overflow; otherwise move rear forward (circularly) and insert.
4. Dequeue: if count = 0, report underflow; otherwise remove at front and move front forward (circularly).
5. Front and Rear: display the elements at front and rear.
6. Display: print count elements starting from front.
7. Repeat until Exit is selected.

B. Queue using linked list

1. Start and set front = rear = NULL.
2. Display the menu: Enqueue, Dequeue, Front, Rear, Display, and Exit.
3. Enqueue: create a node and attach it after rear; it becomes the new rear.
4. Dequeue: if front is NULL, report underflow; otherwise delete the front node and move front to the next node.
5. Front and Rear: display the data of the front and rear nodes.
6. Display: traverse from front to rear.
7. Repeat until Exit is selected.

Program A – Queue Using Circular Array

TURBO C++ • PROGRAM

```
#include <iostream.h>
#include <conio.h>
#define MAX 5
int q[MAX];
int front = 0, rear = -1, count = 0;     // circular queue
void enqueue(int x) {
    if (count == MAX)
        cout << "Queue Overflow";
    else {
        rear = (rear + 1) % MAX;
        q[rear] = x;
        count++;
        cout << "Enqueued " << x;
    }
}
void dequeue() {
    if (count == 0)
        cout << "Queue Underflow";
    else {
        cout << "Dequeued " << q[front];
        front = (front + 1) % MAX;
        count--;
    }
}
void showfront() {
    if (count == 0) cout << "Queue is empty";
    else cout << "Front element: " << q[front];
}
void showrear() {
    if (count == 0) cout << "Queue is empty";
    else cout << "Rear element: " << q[rear];
}
void display() {
    int i;
    if (count == 0)
        cout << "Queue is empty";
    else {
        cout << "Queue (front to rear): ";
        for (i = 0; i < count; i++)
            cout << q[(front + i) % MAX] << " ";
    }
}
void main() {
    int ch, x;
    clrscr();
    do {
        cout << "\n1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit";
        cout << "\nEnter choice: ";
        cin >> ch;
        switch (ch) {
            case 1: cout << "Enter element: "; cin >> x; enqueue(x); break;
            case 2: dequeue();   break;
            case 3: showfront(); break;
            case 4: showrear();  break;
            case 5: display();   break;
            case 6: break;
            default: cout << "Invalid choice";
        }
    } while (ch != 6);
    getch();
}
```

Output – Queue Using Circular Array

TERMINAL • OUTPUT (sample input shown as typed)

```
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 1
Enter element: 10
Enqueued 10
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 1
Enter element: 20
Enqueued 20
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 1
Enter element: 30
Enqueued 30
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 5
Queue (front to rear): 10 20 30
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 2
Dequeued 10
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 3
Front element: 20
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 4
Rear element: 30
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 5
Queue (front to rear): 20 30
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 6
```

Program B – Queue Using Linked List

TURBO C++ • PROGRAM

```
#include <iostream.h>
#include <conio.h>
struct node {
    int data;
    struct node *next;
};
struct node *front = 0, *rear = 0;
void enqueue(int x) {
    struct node *p;
    p = new node;
    p->data = x;
    p->next = 0;
    if (rear == 0)           // queue was empty
        front = rear = p;
    else {
        rear->next = p;
        rear = p;
    }
    cout << "Enqueued " << x;
}
void dequeue() {
    struct node *p;
    if (front == 0)
        cout << "Queue Underflow";
    else {
        p = front;
        cout << "Dequeued " << p->data;
        front = front->next;
        if (front == 0) rear = 0;
        delete p;
    }
}
void showfront() {
    if (front == 0) cout << "Queue is empty";
    else cout << "Front element: " << front->data;
}
void showrear() {
    if (rear == 0) cout << "Queue is empty";
    else cout << "Rear element: " << rear->data;
}
void display() {
    struct node *p;
    if (front == 0)
        cout << "Queue is empty";
    else {
        cout << "Queue (front to rear): ";
        for (p = front; p != 0; p = p->next)
            cout << p->data << " ";
    }
}
void main() {
    int ch, x;
    clrscr();
    do {
        cout << "\n1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit";
        cout << "\nEnter choice: ";
        cin >> ch;
        switch (ch) {
            case 1: cout << "Enter element: "; cin >> x; enqueue(x); break;
            case 2: dequeue();   break;
            case 3: showfront(); break;
            case 4: showrear();  break;
            case 5: display();   break;
            case 6: break;
            default: cout << "Invalid choice";
        }
    } while (ch != 6);
    getch();
}
```

Output – Queue Using Linked List

TERMINAL • OUTPUT (sample input shown as typed)

```
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 1
Enter element: 10
Enqueued 10
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 1
Enter element: 20
Enqueued 20
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 1
Enter element: 30
Enqueued 30
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 5
Queue (front to rear): 10 20 30
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 2
Dequeued 10
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 3
Front element: 20
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 4
Rear element: 30
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 5
Queue (front to rear): 20 30
1.Enqueue 2.Dequeue 3.Front 4.Rear 5.Display 6.Exit
Enter choice: 6
```

Result

Thus, the queue operations are performed successfully using an array and a linked list.

Exercise 5: Binary Tree Traversal

Aim

To construct a binary tree and perform preorder, inorder, and postorder traversals.

Algorithm

1. Start the program.
2. Create the binary tree nodes.
3. In preorder, visit Root, Left, Right.
4. In inorder, visit Left, Root, Right.
5. In postorder, visit Left, Right, Root.
6. Display the traversals and stop.

Program

TURBO C++ • PROGRAM

```
#include <iostream.h>
#include <conio.h>
struct node {
    int data;
    struct node *left;
    struct node *right;
};
struct node *newnode(int x) {
    struct node *p;
    p = new node;
    p->data = x;
    p->left = 0;
    p->right = 0;
    return p;
}
void preorder(struct node *r) {
    if (r != 0) {
        cout << r->data << " ";
        preorder(r->left);
        preorder(r->right);
    }
}
void inorder(struct node *r) {
    if (r != 0) {
        inorder(r->left);
        cout << r->data << " ";
        inorder(r->right);
    }
}
void postorder(struct node *r) {
    if (r != 0) {
        postorder(r->left);
        postorder(r->right);
        cout << r->data << " ";
    }
}
void main() {
    struct node *root;
    clrscr();
    root = newnode(1);
    root->left = newnode(2);
    root->right = newnode(3);
    root->left->left = newnode(4);
    root->left->right = newnode(5);
    root->right->left = newnode(6);
    root->right->right = newnode(7);
    cout << "Preorder:  ";    preorder(root);
    cout << "\nInorder:   ";  inorder(root);
    cout << "\nPostorder: ";  postorder(root);
    getch();
}
```

Output

TERMINAL • OUTPUT (sample input shown as typed)

```
Preorder:  1 2 4 5 3 6 7
Inorder:   4 2 5 1 6 3 7
Postorder: 4 5 2 6 7 3 1
```

Result

Thus, the binary tree traversals are performed successfully.

Exercise 6: Implementation of Shortest Path Algorithm

Aim

To find the shortest distance from a source vertex to all other vertices using Dijkstra's algorithm.

Algorithm

1. Start and represent the graph using an adjacency matrix.
2. Initialize all distances to infinity and the source distance to zero.
3. Select the unvisited vertex with the minimum distance.
4. Update the distances of its adjacent vertices if a shorter path is found.
5. Mark the selected vertex as visited.
6. Repeat until all reachable vertices are processed.
7. Display the shortest distances and stop.

> **Note:** INF is 9999 (the earlier version used 99999) because int is only 16 bits in Turbo C++ and 99999 would overflow.

Program

TURBO C++ • PROGRAM

```
#include <iostream.h>
#include <conio.h>
#define INF 9999          // int is 16-bit in Turbo C++, so keep INF small
void main() {
    int g[5][5] = {
        {0,   10,  5,   3,   INF},
        {10,  0,   2,   1,   INF},
        {5,   2,   0,   9,   2},
        {3,   1,   9,   0,   2},
        {INF, INF, 2,   2,   0}
    };
    int d[5], visited[5];
    int i, j, v, u;

    clrscr();
    for (i = 0; i < 5; i++) {
        d[i] = INF;
        visited[i] = 0;
    }
    d[0] = 0;                         // source vertex is 0

    for (i = 0; i < 5; i++) {
        u = -1;
        for (j = 0; j < 5; j++)     // pick unvisited vertex with least distance
            if (visited[j] == 0 && (u == -1 || d[j] < d[u]))
                u = j;

        if (d[u] == INF)
            break;
        visited[u] = 1;

        for (v = 0; v < 5; v++)     // relax edges of u
            if (visited[v] == 0 && g[u][v] != INF &&
                d[u] + g[u][v] < d[v])
                d[v] = d[u] + g[u][v];
    }

    cout << "Vertex  Distance\n";
    for (i = 0; i < 5; i++)
        cout << i << "       " << d[i] << "\n";
    getch();
}
```

Output

TERMINAL • OUTPUT (sample input shown as typed)

```
Vertex  Distance
0       0
1       4
2       5
3       3
4       5
```

Result

Thus, Dijkstra's algorithm calculates the shortest distances from the source vertex successfully.

Exercise 7: Linear Search

Aim

To search for an element in an array using linear search.

Algorithm

1. Start and read the array elements.
2. Read the element to search.
3. Compare the key with each array element.
4. If a match is found, display its position.
5. Otherwise, display “Element not found.”
6. Stop.

Program

TURBO C++ • PROGRAM

```
#include <iostream.h>
#include <conio.h>
void main() {
    int a[50], n, key, i;
    clrscr();
    cout << "Enter number of elements: ";
    cin >> n;
    cout << "Enter elements: ";
    for (i = 0; i < n; i++)
        cin >> a[i];
    cout << "Enter key: ";
    cin >> key;
    for (i = 0; i < n; i++)
        if (a[i] == key)
            break;
    if (i < n)
        cout << "Element found at position " << i + 1;
    else
        cout << "Element not found";
    getch();
}
```

Output

TERMINAL • OUTPUT (sample input shown as typed)

```
Enter number of elements: 6
Enter elements: 25 10 45 30 15 50
Enter key: 30
Element found at position 4
```

Result

Thus, the required element is searched and its position is displayed successfully.

Exercise 8: Binary Search

Aim

To search for an element in a sorted array using binary search.

Algorithm

1. Start and read the sorted array.
2. Read the key and initialize `low = 0`, `high = n - 1`.
3. Calculate the middle position.
4. If the middle element equals the key, display its position.
5. If the key is greater, search the right half; otherwise, search the left half.
6. Repeat until the key is found or the search range becomes empty.
7. Stop.

Program

TURBO C++ • PROGRAM

```
#include <iostream.h>
#include <conio.h>

void main() {
    int a[50], n, key, i;
    int low, high, mid, found = 0;

    clrscr();
    cout << "Enter number of elements: ";
    cin >> n;
    cout << "Enter sorted elements: ";
    for (i = 0; i < n; i++)
        cin >> a[i];

    cout << "Enter key: ";
    cin >> key;

    low = 0;
    high = n - 1;
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] == key) {
            found = 1;
            break;
        }
        else if (a[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    if (found == 1)
        cout << "Element found at position " << mid + 1;
    else
        cout << "Element not found";
    getch();
}
```

Output

TERMINAL • OUTPUT (sample input shown as typed)

```
Enter number of elements: 7
Enter sorted elements: 10 20 30 40 50 60 70
Enter key: 50
Element found at position 5
```

Result

Thus, binary search identifies the position of an element in a sorted array successfully.

Exercise 9: Insertion Sort

Aim

To sort an array in ascending order using insertion sort.

Algorithm

1. Start and read the array elements.
2. Select the second element as the key.
3. Compare the key with the preceding elements.
4. Shift larger elements one position to the right.
5. Insert the key into its correct position.
6. Repeat for all elements.
7. Display the sorted array and stop.

Program

TURBO C++ • PROGRAM

```
#include <iostream.h>
#include <conio.h>

void main() {
    int a[50], n, key, i, j;

    clrscr();
    cout << "Enter number of elements: ";
    cin >> n;
    cout << "Enter elements: ";
    for (i = 0; i < n; i++)
        cin >> a[i];

    for (i = 1; i < n; i++) {
        key = a[i];
        j = i - 1;
        while (j >= 0 && a[j] > key) {      // shift larger elements right
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }

    cout << "Sorted array: ";
    for (i = 0; i < n; i++)
        cout << a[i] << " ";
    getch();
}
```

Output

TERMINAL • OUTPUT (sample input shown as typed)

```
Enter number of elements: 5
Enter elements: 45 12 8 30 20
Sorted array: 8 12 20 30 45
```

Result

Thus, the array is sorted in ascending order using insertion sort successfully.

Quick Revision Table

Ex.	Topic	Key Concept	Time Complexity
1	Data Types and Conditional Statements	Data types, if-else ladder, grading	—
2	Finding Complexity of a Simple Algorithm	Maximum of an array; counting comparisons	Time: O(n); Space: O(1) (as printed by the program)
3	Stack Using Array and Linked List	Stack — LIFO; array (top index) and linked list (top pointer)	*Not specified in source*
4	Queue Using Array and Linked List	Queue — FIFO; circular array and linked list (front/rear)	*Not specified in source*
5	Binary Tree Traversal	Tree traversals: preorder, inorder, postorder (recursion)	*Not specified in source*
6	Implementation of Shortest Path Algorithm	Dijkstra's algorithm: adjacency matrix, relax edges	*Not specified in source*
7	Linear Search	Linear search: compare one by one	*Not specified in source*
8	Binary Search	Binary search: halve the sorted range	*Not specified in source*
9	Insertion Sort	Insertion sort: shift and insert the key	*Not specified in source*

Complexity values are shown only where the source provides them; the source left the other cells blank, so none have been added.

Turbo C++ 3.0 reminders: use `void main()`, `#include <iostream.h>` and `<conio.h>`; no `using namespace std`, no `bool`, `string` or STL; declare loop variables at the top of main() (a for-loop variable leaks into the enclosing block); int is 16-bit (max 32767), so INF = 9999 in Exercise 6. Use 0 instead of NULL for null pointers.

Verification note: these programs were compile-checked with a standard C++98 compiler using stand-in `iostream.h` and `conio.h` headers, and the sample outputs were captured from real runs. They have not been run in Borland Turbo C++ itself, so test each once in your lab IDE before the exam.